2407012 54-CSE
3rd year

INDEX

NAME: KEERTHANA.G STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: COMPUTER NETWORKS

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
EXP-1	27.7.26	Study of various Linux and windows network commands		27/7/26
EXP-2	27.7.26	Study of network cables and Preparation of straight through and cross-over cables		27/7/26
EXP-3	27.7.26	To understand the packet environment of CISCO packet tracker to design simple network		27/7/26
EXP4.	2.8.26	Design a simple Topology and configure with one router, two switches and 16 PC's using cisco packet tracker.		
EXP5	03.8.26	SETup and config LAN		
EXP6.	11.8.26	Customize switch network modules in CISS packet		
EXP7.	17.8.26	Develop a customized ping command to test remote connectivity		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
Exp8	20.8.26	Anonymous FTP server using python		
Exp 10	24.8.26	Design a simple calculator using XMLRPC		
Exp9	30.8.26	Implement packet sniffing using raw sockets in python		
Ex11	1.9.26	Develop a Program to create reverse TCP connection using python.		
Ex12	1.9.26	Capturing and Analyzing Network Packet using wireshark		
Ex13	5.9.26	Analyze Different types of web Page using URL analysis		
Ex 14	05.9.26	Discover live hosts using Nmap scan on TryHACKME platform		

Exp: 1
27.7.24 Study of Various Network commands used in LINUX and WINDOWS

AIM: To study and practice various networking commands in linux and windows for checking IP configurations.

PROCEDURE:

1. Open terminal in Linux
2. Execute networking commands one by one
3. Observe the output of each command
4. Verify network configuration.
5. Record the purpose and Output
6. Compare similar commands in linux
7. Analyze the results.

Network COMMANDS

1. hostname
Displays the computer name
Syntax: hostname

2. Ping: checks connectivity between two computers using ICMP packets.
syntax: ping google.com , ping <hostname/IP>

3. ipconfig (windows): Displays IP configuration
Syntax: ipconfig/all

4. ip address (Linux): Display IP address
Syntax: IP address show

5. ifconfig (Linux): shows network interface information.
Syntax: ifconfig

6. arp : Displays ARP table (IP to MAC Mapping)

Syntax : arp -a

7. netstat : Displays active network connections and listening ports

Syntax : netstat

8. nslookup : Finds the IP address of a domain

Syntax : nslookup domain_name

9. route : Displays routing table

Syntax : route -n

10. pathping (windows) : checks packet loss and route

Syntax : pathping google.com

11. trace route : shows the path taken by packets

syntax : traceroute google.com

12. mtr : Combination of ping and traceroute

Syntax : mtr gogle .com

Result :

Thus, Various networking commands in Linux and windows were succesfully executed and studied. The IP configuration, Connectivity, routing DNS resolution and network interface information were obtained and verified.

27/7/2026

EXP. 2
22.7.26

STUDY OF NETWORK CABLES AND PREPARATION OF STRAIGHT THROUGH AND CROSS-OVER CABLES

Aim:
- To study different types of network cables used in CN
- To prepare a straight-Through cable and a cross-over cable using an RJ45 Crimping Tool
- To test the cables using a LAN cable tester.

PROCEDURE:

A) Straight-Through Cable
- Cut the UTP cable to the required length
- Remove approximately 2cm of the outer insulation
- Arrange the wires accordly to the T568B color standard
- Trim the wires evenly
- Insert the wires into the RJ45 connector
- Crimp the connector using the crimps tools.
- Repeat the same T568B standard at the other end
- Test the cable using the CAN Cable tester.

B) Cross-Over Cable
- strip both ends of the UTP Cable
- Arrange one end using T568A
- Arrange the other end using T568B
- Insert the wires into the RJ45 Connectors
- Crimp both ends
- Test the Cable using the LAN Cable tester.

COLOR - CODING:

T568A

PIN	COLOR
1	White - Green
2	Green
3	White - orange
4	Blue
5	White - Blue
6	Orange
7	White - Brown
8	Brown

T568B

PIN	COLOR
1	White - Orange
2	Orange
3	White - Green
4	Blue
5	White - Blue
6	Green
7	White - Brown
8	Brown.

OBSERVATION:

Straight - Through code

Both ends are wired using T568B

Cable Tester Output:

1->1
2->2
3->3
4->4
5->5
6->6
7->7
8->8

Status: PASS

Cross - Over Cable

One end uses T568A and the other uses T658B

Cable Tester Output.

1 → 3
2 → 6
3 → 1
4 → 4
5 → 5
6 → 2
7 → 7
8 → 8

STATUS : PASS

Result:

Thus different types of network cables were studied and both straight. Through and Cross-over cables were successfully prepared and tested using the CAN cable tester.

Exp. 3
27-7-26

TO UNDERSTAND THE PACKER ENVIRONMENT OF CISCO PACKET TRACER TO DESIGN SIMPLE NETWORK.

AIM: TO study the CISCO packet Tracer Tool Installation

Introduction:

CISCO Packet tracer is a network simulation software developed by CISCO for designing etc.

FEATURES:

- It allows user to model simple and complex networks
- It helps students practic networking concepts in a virtual environment

INSTALLING CISCO PACKET TRACER.

- Visit Cisco networking Academy (Net Acad)
- Create/ login
- Download ciso Packet tracer
- Install the software
- Launch Cisco Packet Tracer.

USER INTERFACE OVERVIEW:

- Menu BAR - Contains file, edit, options etc.
- Main Toolbar - Provides shortcuts for new, open, save.
- Logical / Physic workspace : Used to switch the logical
- Workspace : - Area where network topology is created.
- User Created Packet Box - Displays packets created during simulation.

PROCEDURE:

Part A: Network using HUB

step1: Open Cisco Packet tracer

step2: From Network Devices - drag:.

step3: Click Connections.

step4: Assign the follow IP addresses.

Device	IP address	Subnet Mask
PC0	10.11.1	255.0.0.0
PC1	10.11.2	255.0.0.0
PC2	10.1.1.3	255.0.0.0
PC3	10.1.1.4	255.0.0.0

step5: Click the add simple PDU (Envelope icon)

step6: Repeat the same process for every PC connected to the hub.

PART B: Network using Switch

Step1: Remove the hub

Step2: Connect all PCs using Copper straight-Through Cable

Step3: Assign the same IP address.

Device	IP address
PC0	10.1.1.
PC1	10.1.1.2
PC2	10.1.1.3
PC3	10.1.1.4

Subnet Mask: 255.0.0.0.

Step4: Click

Add simple PDU

Send packets between the PCs

Observe packet flow

OBSERVATION:

HUB:

* Hub broadcasts every packet to all connected ports.
* Every computer receives the frame
* Only the destination computer accepts the packet
* More collision occur
* Hub doesn't maintain a MAC address table

SWITCH:

* Switch learns MAC address automatically
* Packet is forwarded only to the destination port
* Faster communication
* Less network traffic
* Almost no collisions.

Result:

Thus the cisco Packet tracer environment was studied successfully. A simple network was created using HUB and switch, and the behavior of both devices was analyzed successfully.

27/11/24

Exp: 4
2.8.25

Design a simple topology and configure with one router, two switches and PC, using Cisco Packet Tracer

1) Build the topology:

- Add End Devices

Topology Setup & Connections:

- LAN 1: PC0 (Fa0) to Switch 0 (Fa0/1), PC1 (Fa0) to switch 0
- LA 2: PC2 to swith 1, PC3 to Switch 1 and Switch 1 (Gig0/1)
- to Router 0 (Gig0/0/1)
- Connection cables used: Copper straight-Through

IP Addressing Table

Device	Interface	IP Address	Subnet Mask
PC0	Fast Ethernet 0	192.168.10.10	255.255
PC1	Fast Ethernet 0	192.168.10.11	192.168.10.1
PC2	Fast Ethernet 0	192.168.20.10	192.168.20.1
PC3	Fast Ethernet 0	192.168.20.11	192.168.20.1
Router 0	Gigabit Ethernet	192.168.10.1	N/A
Router 1	GigabitEthernet	192.168.20.1	N/A

Router Commands:

```
Router> enable
Router# configure terminal
Router(config)# interface Gigabit Ethernet 0/0/0
Router(config-if)# exit
Router(config)# interface Gigabit Ethernet 0/0/1
Router(config-if)# ip address 192.168.20.10
Router(config-if)# no shutdown
Router(config-if)# end
Router# write memory
```

Verification Commands:

From PC0's Command Prompt:

- Ping 192.168.10.1
- ping 192.168.20.10

Result:

The network topology was designed and successfully configured. The ping commands returned successful replies, confirming proper routing between LAN1 and LAN2

Exp. 5

23/08/24 SETUP ADD CONFIGURE LAN USING A SWITCH 2 ETHERNET

AIM:

To study and configure a Local Area network (LAN) using a switch and Ethernet

What is LAN?

LAN is a network that connects devices within a limited area, such as home school or office. It allows users to share files, printers, internet access and other resources.

PROCEDURE:

1. Arrange the required devices 4 computers, 1 network switch and ethernet cables

2. Connect each computer to switch using Ethernet cables

3. Configure a static IP address for each PC

PC1 - 10.1.1.1

PC2 - 10.1.1.2

PC3 - 10.1.1.3

PC4 - 10.1.1.4

Subnet Mask — 255.0.0.0

4. Assign the switch the IP address 10.11.5 with subnet mask 255.0.0.0

5. Open cmd and use ping command to test communication between devices

6. share a folder by going to Properties share & allow access to everyone.

7. Test: Access shared folder from other computer to confirm successful communication & resource sharing

RESULT: LAN was successfully setup, devices communicate with each other, file sharing worked.

Exp.6

1.8.26

CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER.

AIM:

To customize a modular switch in Cisco Packet Tracer by adding Fast Ethernet and Gigabit Fiber.

PROCEDURE:

1. Open the Cisco → Packet Tracer and select network Device → Switches → Switch-PT Empty.
2. Add 2 Pcs and 1 Router 2911 to workspace
3. Open the switch → Physical tab → POWER OFF
4. Add modules.
   - PT-SWITCH-NM-1CFE to slot 0, 1, 2
5. Power on the switch and wait for the IOS prompt
6. Configure IP address on the Pcs and use ping to test connectivity.

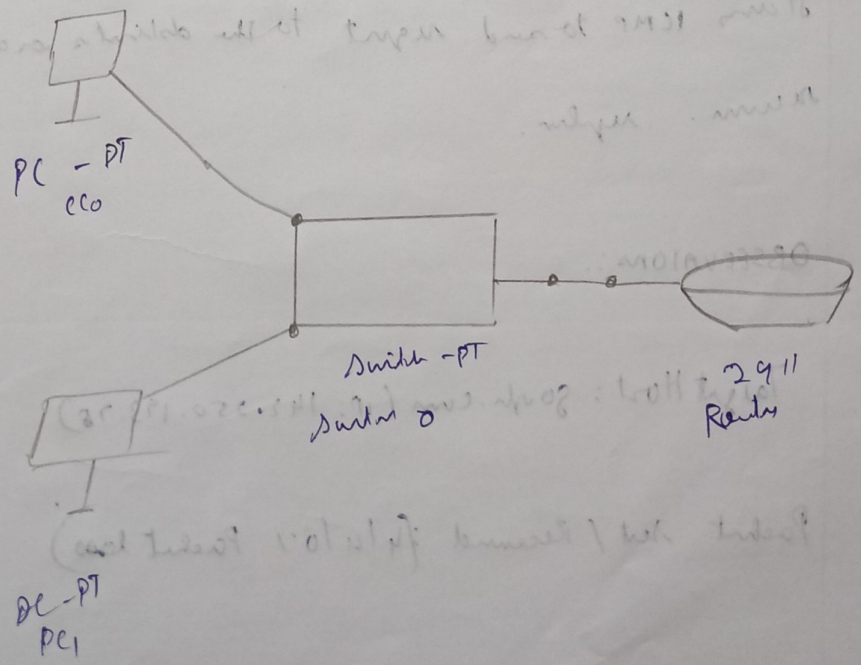
MODULE REFERENCE:

Connection	Module	Cable
PC0 → switch	Fast Ethernet 0/1	CFE
PC1 → switch	Fast Ethernet 1/1	CFE
Router → switch	Gigabit Ethernet	CFE

RESULT:

The connection was done succesfully and the interfaces was verified my CISCO, IOS CFE

OUTPUT:



Exp.7 DEVELOP A CUSTOMIZED PING COMMAND TO TEST SERVER
17.8.26 CONNECTIVITY

AIM:

To develop a simple Python program to check whether a server is reachable using the Ping command.

REQUIREMENTS:

HARDWARE: * Computer
+ Internet

SOFTWARE: Python, VS Code, Command Prompt

THEORY:-

Ping is a network tool used to check whether a server is reachable.

It uses ICMP to send request to the destination and receive reply.

OBSERVATIONS:

Target Host: google.com (IP: 142.250.193.78)

Packet Sent / Received: (4/4/0%. Packet loss)

ROUND TRIP Time (RTT)

- MIN : 27ms
- MAX : 30ms
- Avg : 28 ms

Server status : Reachable.

RESULT:

Thus, a customized ping command was successfully developed in Python to test server connectivity.

EXP: 8

20.8.26

BUILDING AN ANONYMOUS FTP SCANNER USING PYTHON.

AIM:

To develop a python based any FTP scanner using the built-in ftplib module and determine whether access.

REQUIREMENTS:

* windows / Linux
* Python 3.0
* ftplib module
* Authorized FTP server.

PROCEDURE:

* check python installation.
* Create a file named anonymous scanner.py

Import FTP module

Enter FTP server address.

connect to server and list anonymous login

Display accessible directories file.

PROGRAM

```
from ftplib import FTP, all_errors
def anonym(ftp, host, port, out=2):
print("="*40)
print("Anonym FTP scanner")
print("="*40)
ftp(FTP())
print("\n[2] testing anonym login...", ftp.login
("anonym:", "anonym@scanner.com")
print("[+] Anonym login successful.")
print("\n[3] checking dirs...")
files = ftp.nlst()
for item in files:
    print("  -", item)
ftp.quit()
except all_errors as err:
print("[-] Anonym login failed")
print("[!] error:", err)
anonym(ftp, host, port)
```

Result:

FTP server is reachable and allows anonymous access.

Implement packet sniffing using raw sockets in python

AIM:

To study a basic packet sniffer using python that captures network.

PROCEDURE:

1) Install python and add it to the system path

2) Create a project folder and file named sniffer.py

3) Write a program into file

4.) Open command prompt as administrator.

5.) Navigate to the project folder.

6) Run the program.

7) Observe the output

CODE:

```
import socket
import struct

host = socket.gethostbyname(socket.gethostname())

sniffer = socket.socket(socket.AF_INET,

print("Packet sniffer started:")

try:
    while True:
        raw_data, addr = sniffer.recvfrom(65536)
```

Exp. 9

Pg. 9.26

Implement packet sniffer using raw sockets in python

AIM:

To design a basic packet sniffer using python that captures network

PROCEDURE:

1) Install python and add it to the system path

2) Create a project folder and file named sniffer.py

3) Write a program into file

4.) Open command prompt as administrator.

5.) Navigate to the project folder.

6) Run the program.

7) Observe the output

CODE:

```
import socket
import struct

host = socket.gethostbyname(socket.gethostname())

sniffer = socket.socket(socket.AF_INET,

print("Packet sniffer started:")

try:
    while True:
        raw_data, addr = sniffer.recvfrom(65536)
```

Ex. P. 9

24.9.26

Implement packet sniffing using raw sockets in python

AIM:

To develop a basic packet sniffer using python that captures network.

PROCEDURE:

1) Install python and add it to the system path

2) Create a project folder and file named sniffer.py

3) Write a program into file

4.) Open command prompt as administrator.

5.) Navigate to the project folder.

6) Run the program.

7) Observe the output

CODE:

```
import socket
import struct

host = socket.gethostbyname(socket.gethostname())

sniffer = socket.socket(socket.AF_INET,

print("Packet sniffer started:")

try:
    while True:
        raw_data, addr = sniffer.recvfrom(65536)
```

Ex. P. 9

Implement packet sniffing using raw sockets in python

24.9.26

AIM:

To design a basic packet sniffer using python that captures network

PROCEDURE:

1) Install python and add it to the system path

2) Create a project folder and file named sniffer.py

3) Write a program into file

4.) Open command prompt as administrator.

5.) Navigate to the project folder.

6) Run the program.

7) Observe the output

CODE:

```
import socket
import struct

host = socket.gethostbyname(socket.gethostname())

sniffer = socket.socket(socket.AF_INET,

print("Packet sniffer started:")

try:
    while True:
        raw_data, addr = sniffer.recvfrom(65536)
```

Exp. 9

Implement packet sniffer using raw sockets in python

Date: 9.26

AIM:

To study a basic packet sniffer using python that captures network

PROCEDURE:

1) Install python and add it to the system path

2) Create a project folder and file named sniffer.py

3) Write a program into file

4) Open command prompt as administrator.

5) Navigate to the project folder.

6) Run the program.

7) Observe the output

CODE:

```
import socket
import struct

host = socket.gethostbyname(socket.gethostname())

sniffer = socket.socket(socket.AF_INET,

print("Packet sniffer started:")

try:
    while True:
        raw_data, addr = sniffer.recvfrom(65536)
```

```
print ("Version :", ip[0]>>4)
print ("TTL :", pp[5])
print ("Protocol :", ip[6])
print ("Source IP:", socket.inet_ntoa(ip[8:12]))

except KeyboardInterrupt:
    sniffer.ioctl(socket.SIO_RCVALL, socket.
RCVALL_OFF)
    print ("Sniffer stopped")
```

OUTPUT:

Packet sniffer started.

Version 4
TTL : 128
Protocol : 6
Source IP : 192.168.1.5
Destination IP: 192.168.1.1
Packet Length : 60
Sniffer stopped

RESULT:

The python program successfully captures network packets and displays their header information.

EXP: 10

30.8.24 DESIGN a simple calculator using XML RPC

AIM: To design and implementation a remote calculator.

PRE-REQUISITE:
Basic python programming knowledge
understanding of client server.

ALGORITHM / procedure:

SERVER: i) Import simple XML RPC
server
2) Define add subtract, multiply, Divide functions.
3) Create the server on localhost port
4) Register all functions.
5) start the server. serve_forever()

SERVER.PY

from xmlrpc.server import simple

def add(a,b):
    return a+b
def subtract(a,b):
    return a-b
def multiply(a,b):
    return a*b.

def divide(a,b):
    if b==0
    return "Divide by zero not a
server.serve_forever()

CLIENT.PY

import xmlrpc.client

proxy = xmlrpc.client -

a = float(input("Ent
b = float input("Enter
Print("Addition:", proxy
print("subtraction:", p
print("Division:", proxy.

OUTPUT:

XML-RPC calculat

RESULT:

The XML RP
successfully implemen
remotely performed o
server.

```
def divide(a, b):
    if b == 0
    return "Divide by zero not allowed"

server.serve_forever()
```

CLIENT.PY

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8001/")

a = float(input("Enter first number:"))
b = float input("Enter second number:"))

Print("Addition:", proxy.add(a, b))
print("subtraction:", proxy.subtract(a, b))
print("Division:", proxy.divide(a, b))
```

OUTPUT:

XML-RPC calculator server running on port 8000...

RESULT:

The XML RPC remote calculator was successfully implemented and executed. The client remotely performed add, subtract, multiply, divide.

Ex.No:11

1.9.26

DEVELOP a program to create Reliable TCP connection using Python socket:

AIM: To create listen TCP reliable connection using Python between listener on local host

PROCEDURE

Listener Program:

i) Import socket module

ii) set host as 127.0.0.1 and port as 5000

iii) Create a TCP socket using AF-INET and SOCK_STREAM

vi) Join the connection to the server socket

CONNECTOR PROGRAM

i) Set host as 127.00.1 and port as 5000

ii) Create TCP socket and connect to listen.

iii) Send a test message and receive reply

iv) Display reply and close to socket.

PROGRAM CODE:

Listener

import socket

host = "127.0.0.1"

Port = 5000

```
server = socket.socket(socket.AF_INET, socket
    .SOCK_STREAM)
server.listen()

print("Connect to listen...")
client, connect(host, port)
client.send("Hello from connect".encode())
reply = client.recv(1024).decode()
print("Listen replied:", reply)
client.close()
```

LISTENER OUTPUT:

waiting for server connection.

Connection from (127.0.0.1, 54781)

received: Hello from connect.

Connect: output.

Connect to listen.

Listen replied Hello from listen.

Result

TCP server connection established between listen and connect on local host and message was exchanged successfully.

EXP:12 CAPTURING AND Analyzing NETWORK PACKET

USING WIRE SHARK:

9.26

AIM

To capture, filter and analyze network and record the Output.

Description with Procedure:

i) Open wireshark and select the ethernet or local Area, connection interface

ii) Config capture to stop automatically after 100 packets

iii) Start the capture if necessary and save the capture packet

iv) Enter the required protocol filter in the filter bar & input the displayed packet

vi) For flow graph analysis, select statistics → flow Graph.

vi) Repeat the process using following filters.

- TCP : Tcp
- UDP : udp
- ARP : arp
- DNS : dns
- HTTP : http
- IP : ip
- ICMP : icmp
- DHCP : dhcp

viii) Inspect packet details, including source and destination address, protocol fields and packet content. Save the capture for future reference.

Result:

Network packets were captured and analyzed successfully using wireshark. The required protocol filters were applied to identify TCP, UDP, PPP, DNS, HTTP, IP, ICMP, DHCP packet.

EXP:13

15.9.26

Analyze Different Types of web Logs using Webalizer

AIM:

To analyze the different type of web logs using webalize tools

INTRODUCTION:

Webalize is a web server log analyzer tool used to analyze web log files and generate statistical reports. It helps administrators understand website traffic, visitor activity, and server usage.

PROCEDURE:

1. Run The Windows version of the webalize tool
2. Download a sample web log file from the web
3. Open the Webalize tools and provide the downloaded
4. Click the simple webalize option to the process file.
5. Analyze the results.

• Monthly statistics. Displays webat activity for each month, helps to identify changes in traffic over time.

• Host: shows the hosts or IP addresses accessing the website and help analyze.

• User Agents: Provides information about the browser client software used to access the website

OUTPUT:

Webalizer generates contains monthly statistics, hosts and user agents information based web input. The results are displayed in tabular and graphical

RESULT

Thus, different types of web log information were analyzed using webalizer tools.

EXP-14 DISCOVER LIVE HOSTS USING NMAP SCAN ON TRY HACKME PLATFORM

05.9.25

AIM:

To discover live host using NMAP scans (APP/ICMP/TCP) on by machine.

INTRODUCTION:

Nmap is free, open - source tool used to discover active hosts and identify running on a network. Live host discovery helps identify which system are online before performing port scanning. This scans the network and provides running network.

TYPES

i) ARP SCAN : Uses ARP requests to discover active devices on the local network.

ii) ICMP SCAN: Uses ICMP requests to check which hosts are reachable

iii) TCP scan : Uses TCP packets and responses to identify reachable hosts

iv) UDP scan: Uses UDP packets and reports to discover reachable hosts

PROCEDURE

i) Open the TryHackMe and Google

ii) Open the Nmap Live Host Discovery room

using Linux

http://tryhackme.com/room/roomname/original.

iii) start the lab environment if required

iv) Read instruction provided in the room

v) Study ARP scan to discover hosts on local network.

vi) Perform assigned ICMP, based host discovery activity

RESULT:

Thus, live host discovery using Nmap scan was done successfully on tryhackme platform.